

iOS 离屏渲染：识别、解决与实践

一、什么是离屏渲染？

简单来说，就是 GPU 在将内容画到屏幕上之前，需要先把内容画到一个“看不见的屏幕”（离屏缓冲区）上，然后再把结果拷贝回主帧缓冲区。这个额外的步骤会消耗性能，尤其是在滚动列表或动画过程中，会导致掉帧。

二、哪些操作会导致离屏渲染？

1. 圆角 (cornerRadius) + clipsToBounds
2. 阴影 (shadow)
3. 模糊效果 (UIVisualEffectView)
4. 遮罩 (mask)
5. 某些混合模式

三、核心解决思路

核心思想：用 CPU 预处理代替 GPU 的离屏渲染。在内容显示到屏幕上之前，我们就把圆角、阴影等效果“画好”。

四、代码实现案例

案例一：圆角优化（最常见）

✗ 错误示范：直接设置 layer.cornerRadius 和 clipsToBounds

Code block

```
1 // 在 UITableViewCell 的 configure 方法中
2 - (void)configureWithModel:(CellModel *)model {
3     // ✗ 错误：这会触发离屏渲染
4     self.avatarImageView.layer.cornerRadius =
5     self.avatarImageView.frame.size.width / 2.0;
6     self.avatarImageView.layer.masksToBounds = YES;
7
8     // 设置图片
9     self.avatarImageView.image = model.avatarImage;
10 }
```

✓ 正确示范：在图片加载完成时，用 Core Graphics 预合成带圆角的图片

Code block

```
1 // UIImageView+CornerRadius.h
2 #import <UIKit/UIKit.h>
3
4 @interface UIImageView (CornerRadius)
5
6 - (void)setRoundedImage:(UIImage *)image cornerRadius:(CGFloat)cornerRadius;
7
8 @end
9
10 // UIImageView+CornerRadius.m
11 #import "UIImageView+CornerRadius.h"
12 #import <QuartzCore/QuartzCore.h>
13
14 @implementation UIImageView (CornerRadius)
15
16 - (void)setRoundedImage:(UIImage *)image cornerRadius:(CGFloat)cornerRadius {
17     // 1. 使用 Core Graphics 创建一个图形上下文
18     CGSize size = CGSizeMake(image.size.width, image.size.height);
19     UIGraphicsBeginImageContextWithOptions(size, NO, [UIScreen
mainScreen].scale);
20
21     // 2. 在上下文中绘制一个圆形路径
22     UIBezierPath *path = [UIBezierPath bezierPathWithRoundedRect:CGRectMake(0,
0, size.width, size.height) cornerRadius:cornerRadius];
23     [path addClip];
24
25     // 3. 将原始图片绘制到这个圆形区域内
26     [image drawAtPoint:CGPointZero];
27
28     // 4. 从上下文中获取新的、已经裁剪好的图片
29     UIImage *roundedImage = UIGraphicsGetImageFromCurrentImageContext();
30
31     // 5. 结束上下文
32     UIGraphicsEndImageContext();
33
34     // 6. 将处理好的图片设置给 UIImageView
35     self.image = roundedImage;
36
37     // 7. 清除 layer 的圆角设置，因为图片本身已经是圆角的了
38     self.layer.cornerRadius = 0;
39     self.layer.masksToBounds = NO;
40 }
41
```

```

42 @end
43
44 // 在 Cell 中使用
45 - (void)configureWithModel:(CellModel *)model {
46     // ✅ 优化：使用预处理好的圆角图片
47     [self.avatarImageView setRoundedImage:model.avatarImage
48     cornerRadius:self.avatarImageView.frame.size.width / 2.0];
49 }

```

案例二：阴影优化

❌ 错误示范：直接给 layer 添加阴影

Code block

```

1 // 给一个卡片视图添加阴影
2 cardView.layer.shadowOffset = CGSizeMake(0, 2);
3 cardView.layer.shadowOpacity = 0.3;
4 cardView.layer.shadowRadius = 4.0;
5 // 这样设置会触发离屏渲染

```

✅ 正确示范：用一个单独的阴影 UIView，或者用 shadowPath

Code block

```

1 // 方案一：使用 shadowPath (推荐，性能最好)
2 // 在视图布局完成后调用此方法
3 - (void)setupShadowForView:(UIView *)view {
4     view.layer.shadowOffset = CGSizeMake(0, 2);
5     view.layer.shadowOpacity = 0.3;
6     view.layer.shadowRadius = 4.0;
7
8     // ★ 关键：手动指定阴影的形状，避免 GPU 自动计算
9     view.layer.shadowPath = [UIBezierPath
10     bezierPathWithRect:view.bounds].CGPath;
11     // 或者，如果是圆角矩形
12     // view.layer.shadowPath = [UIBezierPath
13     bezierPathWithRoundedRect:view.bounds cornerRadius:cornerRadius].CGPath;
14 }
15
16 // 方案二：使用独立的阴影视图
17 // 创建一个与卡片大小一致的视图作为阴影
18 UIView *shadowView = [[UIView alloc] init];
19 shadowView.backgroundColor = [UIColor colorWithWhite:0 alpha:0.3];
20 shadowView.layer.cornerRadius = 8.0; // 与卡片圆角保持一致
21 [self.view insertSubview:shadowView belowSubview:cardView];

```

五、优秀的第三方库

工具/库	核心功能	特点
Instruments (Core Animation)	性能检测	最权威的工具。在 Core Animation 模式下勾选 "Color Offscreen-Rendered Yellow"，触发离屏渲染的地方会变为黄色，让你一目了然。
YYWebImage	图片加载	一个强大的图片加载库，其作者对性能优化有深刻见解，代码中包含了一些规避离屏渲染的实践。
GPU Image	图像处理	如果你需要复杂的实时滤镜或图像处理，它使用 OpenGL ES，能高效地在 GPU 端完成，但这通常用于特殊场景。

总结与建议

1. 检测是第一步：使用 Instruments 的 "Color Offscreen-Rendered Yellow" 功能，找到项目中所有离屏渲染的点。
2. 核心是 CPU 预处理：对于圆角、圆图等常见问题，用 Core Graphics 预处理是标准解决方案。
3. 优化 shadowPath：对于阴影，设置 shadowPath 是最简单且有效的优化手段。
4. 权衡利弊：CPU 预处理会占用 CPU 时间。对于静态内容（如用户头像），一次处理，长期使用，非常划算。对于频繁变化的动态内容，需要仔细权衡。
5. 分工协作：用 SDWebImage 来解决“下载”和“缓存”问题；用自己写的 Core Graphics 代码来解决“圆角”等导致的“离屏渲染”问题。

（注：文档部分内容可能由 AI 生成）